

Compressione delle immagini con l'algoritmo Run-Length Encoding (RLE)

Le immagini digitali sono composte da **pixel**, cioè piccoli quadrati che rappresentano un colore. In alcune immagini, soprattutto quelle semplici (ad esempio immagini in bianco e nero o con grandi aree dello stesso colore), capita che **molti pixel consecutivi abbiano lo stesso valore**.

Per ridurre lo spazio necessario a salvare queste immagini si possono usare algoritmi di **compressione dei dati**. Uno dei più semplici è il **Run-Length Encoding (RLE)**.

Idea dell'algoritmo

Invece di memorizzare tutti i valori dei pixel uno per uno, l'algoritmo memorizza:

- **il valore**
- **quante volte si ripete consecutivamente**

Esempio:

Sequenza di pixel:

bianco bianco bianco bianco nero nero nero bianco

Compressione RLE:

(white, 4) (black, 3) (white, 1)

In questo modo, quando ci sono molte ripetizioni consecutive, si risparmia spazio.

Questo tipo di compressione è stato utilizzato in diversi formati di immagine, tra cui **BMP image format**, **TIFF image format** e **PCX image format**.

Naturalmente, per ricostruire l'immagine originale basta **ripetere ogni valore per il numero di volte indicato**.

Esercizio compressione

Per capire il funzionamento dell'algoritmo RLE, realizziamo una versione semplificata che lavora **su stringhe invece che su immagini**.

Obiettivo

Scrivi un programma che **comprime una stringa utilizzando l'idea del Run-Length Encoding**.

Il programma deve:

1. leggere una stringa contenente caratteri
 2. individuare i **caratteri consecutivi uguali**
 3. costruire una **lista di liste** in cui ogni sotto-lista contiene:
 - il carattere
 - il numero di volte che compare consecutivamente
- **Esempio Input:** "aaabbc"
 - **Esempio Output:** [['a', 3], ['b', 2], ['c', 1]]

Esercizio decompressione

Scrivi anche un secondo programma che faccia l'operazione inversa (**decompressione**), dato input la lista di liste in cui ogni sotto-lista contiene carattere/numero volte che compare, ricostruire la sequenza originale

attenzione: usare eval() oppure cambiare formato input in esempio

a3b5f6g7

aaabbbbfffffgggggg